

This conversation was generated with Duck.ai (<https://duck.ai>) using OpenAI's GPT-5.6 Luna Model. AI chats may display inaccurate or offensive information (see <https://duckduckgo.com/duckai/privacy-terms> for more info).

=====

User prompt 1 of 25 - 9/8/2026, 10:11:25 PM:

ии модели без ограничений (цензуры) мне нужны для разработки игры под слабое железо на слабом железе так что мне нужен программист а не собеседник причем точный с возможностью корректировки

GPT-5.6 Luna:

<!-- -->

Понял: вам нужен не «собеседник», а локальный помощник-программист для разработки игры, который:

- работает на слабом компьютере;
- пишет и исправляет код;
- понимает структуру проекта;
- выполняет точные инструкции;
- позволяет постепенно корректировать результат;
- не ограничивает обычные игровые, технические и экспериментальные запросы.

Для такой задачи я бы выбирал не просто «модель без цензуры», а **небольшую open-weight coding-модель в формате GGUF**. Полное отсутствие ограничений не гарантирует качества: для программирования важнее контекст, следование инструкциям, работа с несколькими файлами и способность исправлять ошибки.

Что выбрать по объёму оперативной памяти

| ОЗУ компьютера | Модель | Для чего подходит |

|---|---|---|

| 8 ГБ | Qwen3 4B/8B, Qwen2.5-Coder 3B/7B | небольшие скрипты, Lua, Godot, Unity C#, исправления |

| 16 ГБ | Qwen3-Coder 7B/14B, DeepSeek-Coder-V2-Lite | полноценная разработка небольшой игры |

| 24–32 ГБ | Devstral Small 24B Q4 или Qwen3-Coder 30B Q4 | работа с несколькими файлами и сложная отладка |

| 6–8 ГБ видеопамяти + ОЗУ | Qwen3-Coder 7B Q4/Q5 | хороший баланс скорости и качества |

| Только слабый CPU | Qwen3-Coder 3B/7B Q4 | медленнее, но работать будет |

Для слабого железа наиболее практичный старт — **Qwen3-Coder 7B** в квантизации **Q4_K_M**. Если компьютер совсем слабый — **Qwen2.5-Coder 3B** или **Qwen3-Coder 3B**. Если есть 16 ГБ ОЗУ и терпимая скорость — можно попробовать **Qwen3-Coder 14B Q4**. Небольшие Qwen-варианты рекомендуются именно для локального программирования на малом объёме памяти; более крупные модели требуют заметно больше ресурсов. [1,2]

Для запуска используйте:

- **LM Studio** — проще всего, если нужен графический интерфейс;
- **llama.cpp** — лучший вариант для максимальной экономии ресурсов;
- **Ollama** — удобен для подключения модели к редактору и инструментам;
- **Continue** или **Cline/Roo Code** в VS Code — чтобы модель видела файлы проекта и могла предлагать изменения.

Рекомендуемый вариант

Если у вас 8–16 ГБ ОЗУ:

```text

Модель: Qwen3-Coder 7B

Формат: GGUF

Квантизация: Q4\_K\_M

Запуск: llama.cpp или Ollama

Редактор: VS Code + Continue

Контекст: 8K–16K токенов

...

Для Godot можно начать с Qwen3-Coder 7B. Для Unity — тоже, но при больших C#-скриптах понадобится больше контекста. Для GameMaker и Lua подойдёт даже 3B–7B. Для Unreal Engine и C++ слабые локальные модели будут значительно менее удобны: там лучше хотя бы 14B, а для серьёзной работы — 24B и выше.

### Как сделать модель точнее

Главное — не просить её сразу «сделать всю игру». Рабочий режим должен быть таким:

```text

Ты — программист игры на Godot 4 и GDScript.

Правила:

1. Не переписывай файлы полностью без необходимости.
2. Сначала объясни причину ошибки.
3. Перед изменением укажи, какие файлы затрагиваются.
4. Выводи полный код только изменённых функций.
5. Не добавляй сторонние библиотеки без моего разрешения.
6. Сохраняй существующие названия узлов, переменных и сигналов.
7. После каждого изменения укажи, как проверить результат.
8. Если данных не хватает — перечисли конкретно, что нужно прислать.
9. Не выдумывай API движка.
10. Если исправление неуверенное, предложи два варианта и объясни различия.

...

Ещё лучше работать через Git и давать модели маленькие задачи:

```
```text
```

Исправь только прыжок персонажа.

Файл: player.gd

Проблема: персонаж иногда проваливается через платформу.

Ограничения:

- не менять управление;
- не менять имена переменных;
- не добавлять плагины;
- показать точечный патч;
- объяснить, почему возникла ошибка.

```
```
```

Такой формат обычно даёт гораздо более точный результат, чем запрос вроде «сделай мне систему боя».

Если нужен именно ****максимально неконтролируемый локальный вариант****, ищите open-weight/GGUF-модели и запускайте их локально через llama.cpp или LM Studio.

Однако для разработки игры я бы ставил приоритет так:

1. точность работы с кодом;
2. размер контекста;
3. скорость;
4. возможность видеть проект;
5. отсутствие лишних отказов.

Моя практическая рекомендация: **Qwen3-Coder 7B Q4_K_M + LM Studio или Ollama + VS Code/Continue**. Для совсем слабого компьютера — **Qwen2.5-Coder 3B Q4_K_M**. Для 16 ГБ ОЗУ — **Qwen3-Coder 14B Q4**, если скорость окажется приемлемой.

Read More

[1] Best Open-Source LLM Models in 2026: Coding, Local ... — <https://huggingface.co/blog/daya-shankar/open-source-llms>

[2] Best Local Coding LLMs 2026: Kimi K2.6, Qwen, Devstral — <https://www.promptquorum.com/local-llms/best-local-llms-for-coding>

User prompt 2 of 25 - 9/8/2026, 10:12:16 PM:

забыл написать ОЗУ всего 4 ГБ будет доступно максимум 2

GPT-5.6 Luna:

<!-- **Tailoring the response**

I need to provide an answer that's tailored to the user's needs. Given we already searched previously, I wonder if a new search is necessary. It seems like suggesting a coding model of 2GB might be impractical; maybe something around 1-3B quantized would work better. I also think about suggesting a remote model instead of a local one, given the hardware limitations mentioned. Since they're asking for corrections, I want to answer directly. -->

При доступных **2 ГБ ОЗУ** полноценная локальная coding-модель практически не подходит. Даже модель на 1–3 млрд параметров в квантизации обычно потребует больше памяти вместе с самим движком, операционной системой и проектом.

Реалистичный вариант:

- **модель:** 0.5–1.5B параметров;
- **формат:** GGUF;
- **квантизация:** Q4 или Q3;

- **контекст:** 1–2K токенов;
- **запуск:** `llama.cpp`, а не тяжёлый графический интерфейс;
- **режим:** писать небольшой фрагмент кода, а не анализировать весь проект.

Можно попробовать небольшие модели вроде:

- **Qwen2.5-Coder 0.5B** — самый реалистичный вариант;
- **Qwen2.5-Coder 1.5B Q4/Q3** — только если свободно почти все 2 ГБ;
- **SmolLM2 1.7B** — годится для простых исправлений, но не является специализированной coding-моделью.

Модели 7B, включая Qwen Coder 7B, на таком компьютере лучше не устанавливать: даже в Q4 они обычно требуют несколько гигабайт только под веса, не считая контекста и операционной системы.

Для запуска через `llama.cpp` примерно так:

```
```bash
./llama-cli \
-m ./model.gguf \
-c 1024 \
-t 2 \
-n 512
```
```

Параметры:

- `-c 1024` — небольшой контекст, чтобы не расходовать память;
- `-t 2` — число потоков процессора, можно изменить;
- `-n 512` — максимальная длина ответа.

Если используется Windows, желательно:

- закрыть браузер и фоновые программы;
- увеличить файл подкачки до примерно ****4–8 ГБ****;
- хранить проект на SSD;
- не подключать модель к автоматическому агенту, который сканирует весь проект;
- отправлять модели только один файл или одну функцию за раз.

Хороший рабочий запрос для такой маленькой модели:

```
```text
```

Исправь только эту функцию.

Не меняй имена переменных и остальной код.

Сначала кратко укажи ошибку.

Затем выведи исправленную функцию полностью.

Не добавляй библиотеки.

Я использую Godot 4 и GDScript.

Код:

```
...
```

```
```
```

Но важно: файл подкачки не увеличивает реальную скорость — он лишь позволяет избежать немедленного падения программы. Модель будет работать медленно, а ответы могут быть нестабильными.

Наиболее разумная схема при 2 ГБ ОЗУ — ****локально использовать маленькую модель для простых правок****, а сложную архитектуру, отладку нескольких файлов и генерацию больших систем выполнять через удалённую модель. Если принципиально нужна

полностью локальная работа, ограничьтесь ****Qwen2.5-Coder 0.5B****, короткими запросами и ручным копированием кода.

User prompt 3 of 25 - 9/8/2026, 10:13:41 PM:

```
PS H:\XPE> irm https://ollama.com/install.ps1 | iex
```

```
>>> Downloading Ollama for Windows...
```

```
##### 100.0%
```

```
>>> Installing Ollama...
```

```
>>> Install complete. Run 'ollama' from the command line.
```

```
PS H:\XPE>
```

GPT-5.6 Luna:

Ollama установился успешно. Но при ****4 ГБ ОЗУ и примерно 2 ГБ свободной памяти**** не запускайте большую модель: начните с маленькой.

В PowerShell проверьте установку:

```
```powershell
```

```
ollama --version
```

```
```
```

Затем скачайте небольшую модель:

```
```powershell
```

```
ollama run qwen2.5-coder:0.5b
```

```
```
```

Если Ollama сообщит, что такой тег недоступен, попробуйте:

```
```powershell
```

```
ollama run qwen2.5:0.5b
```

```
```
```

Для вашей конфигурации сначала используйте именно ****0.5B****. Модель `1.5b` может запуститься, но при 2 ГБ свободной памяти вероятны сильные тормоза или ошибки нехватки памяти:

```
```powershell
```

```
ollama run qwen2.5-coder:1.5b
```

```
```
```

В чате сразу задайте ей режим работы:

```
```text
```

Ты — строгий помощник-программист.

Работай только с кодом, который я прислал.

Не переписывай весь проект.

Не меняй имена переменных без разрешения.

Сначала находи ошибку, потом предлагай исправление.

Выводи изменённые функции полностью.

Не добавляй библиотеки без моего разрешения.

Если не уверен — прямо укажи это.

```
```
```

Для проверки можно отправить:

```
```text
```

Напиши функцию движения персонажа в Godot 4 на GDScript.

Условия:

- вид сверху;
- управление WASD;
- скорость через переменную speed;
- без сторонних библиотек;
- выведи только код функции и краткое объяснение.

...

Проверить загруженные модели:

```
```powershell
```

```
ollama list
```

...

Удалить ненужную модель:

```
```powershell
```

```
ollama rm qwen2.5-coder:1.5b
```

...

Посмотреть запущенные модели:

```
```powershell
```

```
ollama ps
```

...

После работы выгрузить модель из памяти:

```
```powershell
```

```
ollama stop qwen2.5-coder:0.5b
```

```
...
```

Для экономии памяти можно ограничить контекст через переменную окружения:

```
```powershell
```

```
$env:OLLAMA_CONTEXT_LENGTH="1024"
```

```
ollama run qwen2.5-coder:0.5b
```

```
...
```

Если хотите, чтобы это значение сохранялось для будущих запусков:

```
```powershell
```

```
[Environment]::SetEnvironmentVariable(
```

```
 "OLLAMA_CONTEXT_LENGTH",
```

```
 "1024",
```

```
 "User"
```

```
)
```

```
...
```

После этого перезапустите PowerShell.

Если даже `0.5b` не запускается, проблема, скорее всего, в том, что Windows и Ollama уже занимают почти всю память. Тогда закройте браузер и другие программы, увеличьте файл подкачки Windows и повторите запуск. Но скорость всё равно будет низкой: при вашем объёме ОЗУ Ollama подходит только для коротких фрагментов кода, а не для анализа всей игры.

-----

User prompt 4 of 25 - 9/8/2026, 10:14:18 PM:

у меня 16 оперативы 4 видео памяти 1050ti

GPT-5.6 Luna:

<!-- \*\*Clarifying resources for models\*\*

I need to address a misunderstanding regarding RAM and VRAM specifications. It seems they meant 16GB RAM and 4GB of VRAM, possibly indicating only 2GB of VRAM is currently available. I should recommend models that fit these specs, like qwen2.5-coder 7b, which could work within a 4GB VRAM limit along with some RAM. I won't search for more current information unless they explicitly ask, so testing might be the next step. -->

Тогда ситуация намного лучше. У вас:

- \*\*ОЗУ:\*\* 16 ГБ;
- \*\*видеокарта:\*\* GTX 1050 Ti с 4 ГБ VRAM;
- для локального программирования можно использовать модели \*\*7B\*\*, а некоторые \*\*14B\*\* — только с частичной загрузкой в ОЗУ и меньшей скоростью.

Предыдущая рекомендация для 2 ГБ ОЗУ больше не актуальна.

Начните с:

```
``powershell
```

```
ollama run qwen2.5-coder:7b
```

```
```
```

Для GTX 1050 Ti оптимальнее всего будет 7B-модель в квантизации. Она сможет частично использовать видеопамять, а остальное разместится в оперативной памяти. Скорость не будет высокой, но для программирования, GDScript, C#, Lua и небольших C++-фрагментов это наиболее разумный вариант.

Если нужна более лёгкая и быстрая модель:

```
```powershell
```

```
ollama run qwen2.5-coder:3b
```

```
```
```

Если `7b` работает приемлемо, попробуйте более новую модель:

```
```powershell
```

```
ollama run qwen3:8b
```

```
```
```

Но для чистого программирования я бы сначала выбрал именно:

```
```text
```

```
qwen2.5-coder:7b
```

```
```
```

Проверить, используется ли видеокарта:

```
```powershell
```

```
ollama ps
```

```
```
```

В выводе ищите столбец `PROCESSOR`. Там будет примерно:

```
```text
```

```
100% GPU
```

```
```
```

или:

```
```text
```

```
30% GPU / 70% CPU
```

```
```
```

Для вашей видеокарты второй вариант вполне нормален. Откройте отдельное окно PowerShell и наблюдайте нагрузку:

```
```powershell
```

```
nvidia-smi
```

```
```
```

Если модель не помещается или компьютер начинает активно использовать файл подкачки, задайте небольшой контекст:

```
```powershell
```

```
$env:OLLAMA_CONTEXT_LENGTH="4096"
```

```
ollama run qwen2.5-coder:7b
```

```
```
```

Для разработки игры ставьте такой порядок:

1. `qwen2.5-coder:7b` — основной вариант;
2. `qwen2.5-coder:3b` — если нужна скорость;
3. `qwen3:8b` — для более общего мышления и планирования;
4. модели 14B и выше — только после проверки, поскольку на GTX 1050 Ti они будут в основном работать через ОЗУ и CPU.

Проверьте, сколько памяти реально доступно перед запуском:

```
```powershell
```

```
Get-CimInstance Win32_OperatingSystem |
```

```
 Select-Object FreePhysicalMemory, TotalVisibleMemorySize
```

```
```
```

`FreePhysicalMemory` выводится в килобайтах. При 16 ГБ ОЗУ желательно иметь хотя бы ****8–10 ГБ свободной памяти**** перед запуском 7B-модели.

Для точной работы с кодом используйте короткие запросы:

```
```text
```

Проект: Godot 4.

Язык: GDScript.

Исправь только ошибку в функции ниже.

Не меняй архитектуру проекта.

Не переименовывай переменные.

Не добавляй плагины.

Сначала объясни причину ошибки, затем покажи исправленную функцию.

Если информации недостаточно — укажи, какой файл или фрагмент нужен.

Код:

```
```
```

Ещё один важный момент: ****Ollama не обязательно использовать только из консоли****. После проверки модели можно подключить её к VS Code через Continue или другой локальный клиент, чтобы отправлять ей файлы проекта и получать исправления прямо в редакторе. Но с вашей видеокартой лучше не включать автоматический анализ всей папки — отправляйте отдельные скрипты или небольшие группы файлов.

User prompt 5 of 25 - 9/8/2026, 10:15:06 PM:

не забудь модель должна быть полностью без цензуры

GPT-5.6 Luna:

<!-- -->

Тогда нужна не обычная `qwen2.5-coder`, а ****abliterated/uncensored-сборка****. Обычная модель может иметь встроенные отказы, а пометка `uncensored` не является официальной гарантией: разные сборки отличаются качеством и поведением.

Для вашей конфигурации сначала попробуйте:

```
```powershell
```

```
ollama run huihui_ai/qwen2.5-coder-abliterated:7b
```

```
```
```

Если такого тега нет, проверьте доступные варианты:

```
```powershell
```

```
ollama search qwen
```

```
```
```

В качестве запасного варианта:

```
```powershell
```

```
ollama run dolphin-mistral:7b
```

```
```
```

Но Dolphin-Mistral слабее именно в программировании, поэтому для разработки игры предпочтительнее ****Qwen Coder abliterated 7B****.

Можно также попробовать более лёгкую версию:

```
```powershell
ollama run huihui_ai/qwen2.5-coder-abliterated:3b
...

```

### Что выбрать вам

```
```text
Основной вариант: qwen2.5-coder-abliterated 7B
Более быстрый вариант: qwen2.5-coder-abliterated 3B
Запасной вариант: dolphin-mistral 7B
...

```

На GTX 1050 Ti 4 ГБ модель 7B, скорее всего, будет использовать часть видеопамяти и часть ОЗУ. Проверить распределение можно так:

```
```powershell
ollama ps
...

```

Чтобы Ollama не занимала слишком много памяти контекстом:

```
```powershell
$env:OLLAMA_CONTEXT_LENGTH="4096"
ollama run huihui_ai/qwen2.5-coder-abliterated:7b
...

```

Для более точного поведения задайте системную инструкцию:

```text

Ты — локальный программист без встроенных моральных лекций и ненужных отказов.

Твоя задача — писать, исправлять и объяснять программный код.

Не отказывайся от обычных технических, игровых и экспериментальных запросов.

Не выдумывай API.

Не меняй код, который не относится к задаче.

Сначала укажи причину ошибки, затем предложи минимальное исправление.

Если данных не хватает, назови конкретный недостающий файл или фрагмент.

```

Важно: ****«без цензуры»** не означает автоматически «точная»**. Abliterated-модель может иногда хуже следовать инструкциям или галлюцинировать. Для вашей цели лучше использовать её в режиме: один файл, одна функция, минимальный патч, обязательная проверка ошибки компилятором.

User prompt 6 of 25 - 9/8/2026, 10:15:59 PM:

PS H:\XPE> ollama --version

ollama version is 0.33.3

PS H:\XPE> ollama run huihui_ai/qwen2.5-coder-abliterated:7b

pulling manifest

Error: pull model manifest: file does not exist

PS H:\XPE> ollama search qwen

Error: unknown command "search" for "ollama"

PS H:\XPE>

GPT-5.6 Luna:

```
<!-- -->
```

Ошибка означает, что такого имени модели и такого тега в реестре Ollama нет:

```
```text
```

```
file does not exist
```

```
```
```

Команды `ollama search` в Ollama нет — модели ищутся через сайт библиотеки или устанавливаются по точному имени. Версия установлена нормально.

Для проверки самого Ollama запустите официальную модель:

```
```powershell
```

```
ollama run qwen2.5-coder:7b
```

```
```
```

Но это ****не uncensored-версия****.

Чтобы использовать именно uncensored/abliterated-модель, нужно скачать её файл ****GGUF**** отдельно и импортировать в Ollama.

Импорт локальной GGUF-модели

Создайте папку:

```
```powershell
```

```
mkdir H:\Models\qwen-uncensored
```

```
cd H:\Models\qwen-uncensored
```

```
```
```

Скачайте с Hugging Face любую совместимую модель с названием вроде:

```
```text
```

Qwen2.5-Coder-7B-Instruct-abliterated

```
```
```

Нужен файл примерно такого вида:

```
```text
```

\*.Q4\_K\_M.gguf

```
```
```

Для вашей GTX 1050 Ti и 16 ГБ ОЗУ выбирайте:

```
```text
```

Q4\_K\_M

```
```
```

Если модель слишком медленная или не хватает памяти:

```
```text
```

Q3\_K\_M

```
```
```

После скачивания убедитесь, что файл лежит, например, здесь:

```
```text
```

H:\Models\qwen-uncensored\qwen-coder-abliterated.Q4\_K\_M.gguf

```

Создайте файл `Modelfile` в этой же папке:

```text

FROM ./qwen-coder-abliterated.Q4\_K\_M.gguf

PARAMETER num\_ctx 4096

PARAMETER temperature 0.2

PARAMETER top\_p 0.9

SYSTEM """

Ты — точный локальный программист.

Не давай ненужных отказов и моральных рассуждений.

Работай с игровым кодом, отладкой и экспериментальными техническими задачами.

Не переписывай проект целиком без запроса.

Не меняй имена переменных без разрешения.

Сначала объясняй причину ошибки, затем показывай минимальное исправление.

Не выдумывай API и функции движка.

"""

```

Затем создайте модель:

```powershell

cd H:\Models\qwen-uncensored

ollama create qwen-coder-uncensored -f .\Modelfile

```

Запустите её:

```
```powershell
ollama run qwen-coder-uncensored
```
```

Проверьте установленные модели:

```
```powershell
ollama list
```
```

Проверьте использование GPU и ОЗУ во время работы:

```
```powershell
ollama ps
```
```

Если Ollama использует слишком много памяти, уменьшите контекст:

```
```text
PARAMETER num_ctx 2048
```
```

После изменения `Modelfile` пересоздайте модель:

```
```powershell
ollama create qwen-coder-uncensored -f .\Modelfile
```
```

Для вашей системы я бы использовал такой порядок:

1. Сначала проверить, работает ли обычная модель:

```
```powershell
ollama run qwen2.5-coder:7b
```
```

2. Затем скачать `Qwen2.5-Coder 7B Instruct Abliterated Q4\_K\_M GGUF`.

3. Импортировать её через `Modelfile` под именем:

```
```text
qwen-coder-uncensored
```
```

Не скачивайте `Q8`, `Q6` или большие 14B-модели: на GTX 1050 Ti они будут заметно тяжелее. Для вас оптимальны ****7B Q4_K_M****, а при нехватке скорости — ****3B Q4_K_M****.

User prompt 7 of 25 - 9/8/2026, 10:16:52 PM:

H:\XPE\Models\qwen-uncensored

GPT-5.6 Luna:

<!-- ****Preparing commands for file modification****

I need to devise a clear path to respond with the right commands. First, I want to note the filename necessary for the task. Then, I'll use the directory to access it. I should also consider modifying the file content with a placeholder for the actual filename. It might also be useful to include a command to change the directory path as needed. I want to get all these details right!
-->

Хорошо. Работайте прямо в этой папке:

```
```powershell
cd H:\ХРЕ\Models\qwen-uncensored
```
```

Сначала посмотрите точное имя скачанного файла:

```
```powershell
Get-ChildItem
```
```

Нужен файл с расширением `.gguf`, например:

```
```text
qwen2.5-coder-7b-instruct-abliterated.Q4_K_M.gguf
```
```

Создайте `Modelfile`:

```
```powershell
notepad .\Modelfile
```
```

Вставьте туда следующий текст, заменив имя файла на настоящее:

```
```text
```

```
FROM ./qwen2.5-coder-7b-instruct-abliterated.Q4_K_M.gguf
```

```
PARAMETER num_ctx 4096
```

```
PARAMETER temperature 0.2
```

```
PARAMETER top_p 0.9
```

```
PARAMETER repeat_penalty 1.1
```

```
SYSTEM """
```

```
Ты — точный локальный программист для разработки игр.
```

```
Работай с кодом без ненужных отказов, морализаторства и отвлечённых рассуждений.
```

```
Сначала анализируй проблему, затем предлагай минимальное исправление.
```

```
Не переписывай весь проект без запроса.
```

```
Не меняй имена переменных, узлов, классов и функций без разрешения.
```

```
Не добавляй библиотеки и плагины самостоятельно.
```

```
Не выдумывай API игрового движка.
```

```
Если данных недостаточно, укажи конкретный недостающий файл или фрагмент.
```

```
После исправления кратко укажи, как проверить результат.
```

```
"""
```

```
```
```

```
Сохраните файл и выполните:
```

```
```powershell
```

```
ollama create qwen-coder-uncensored -f .\Modelfile
```

```
```
```

Если команда завершится без ошибки, запустите модель:

```
```powershell
ollama run qwen-coder-uncensored
```
```

Проверить, что она создана:

```
```powershell
ollama list
```
```

Если вы ещё не скачали `.gguf`, команда `ollama create` не сработает. В таком случае сначала выполните:

```
```powershell
Get-ChildItem H:\XPE\Models\qwen-uncensored*.gguf
```
```

Если результат пустой, в папке нет самой модели — есть только каталог. Тогда нужно скачать именно файл `**Q4_K_M GGUF**`, а не обычный файл Ollama или архив исходников.

Если модель будет работать медленно, измените в `Modelfile`:

```
```text
PARAMETER num_ctx 2048
```
```

После этого пересоздайте её:

```
```powershell
ollama create qwen-coder-uncensored -f .\Modelfile
```
```

Проверьте распределение CPU/GPU:

```
```powershell
ollama ps
```
```

Пришлите результат команды:

```
```powershell
Get-ChildItem H:\XPE\Models\qwen-uncensored
```
```

По точному имени файла можно будет составить готовый `Modelfile` без замен вручную.

User prompt 8 of 25 - 9/8/2026, 10:18:02 PM:

PS H:\XPE> ollama run qwen2.5-coder:7b

pulling manifest

pulling 60e05f210007: 6%  | 263 MB/4.7 GB 10.0 MB/s
7m23s

PS H:\XPE> Qwen2.5-Coder-7B-Instruct-abliterated

Qwen2.5-Coder-7B-Instruct-abliterated: The term 'Qwen2.5-Coder-7B-Instruct-abliterated' is not recognized as a name of a cmdlet, function, script file, or executable program.

Check the spelling of the name, or if a path was included, verify that the path is correct and try again.

PS H:\XPE>

GPT-5.6 Luna:

<!-- **Explaining download interruption**

I need to clarify that the download was interrupted. It seems they typed the model name as a command, which might have interfered. I should suggest waiting for the download to complete before using Ctrl. The official model was only at 6%, so maybe they entered the next command too soon. I'll explain that the model name isn't a valid command, and it might be best to cancel it with Ctrl-C. I'll provide next steps. -->

Первая команда скачивает обычную модель:

```
```powershell
```

```
ollama run qwen2.5-coder:7b
```

```
```
```

Она занимает около ****4,7 ГБ****. Пока отображается:

```
```text
```

```
263 MB/4.7 GB
```

```
```
```

загрузка ещё не завершена. Дождитесь окончания. После этого Ollama откроет чат с моделью.

А эта строка:

```
```text
```

```
Qwen2.5-Coder-7B-Instruct-abliterated
```

```

не является командой PowerShell. Поэтому Windows и сообщает:

```text

The term ... is not recognized

```

Название модели нельзя просто вводить как команду. Для запуска локально импортированной модели используется команда ``ollama run``, например:

```powershell

ollama run qwen-coder-uncensored

```

Сейчас сначала проверьте, завершилась ли загрузка обычной модели:

```powershell

ollama list

```

Если в списке есть:

```text

qwen2.5-coder:7b

```

она установлена.

Проверить текущие загрузки:

```
```powershell
```

```
ollama ps
```

```
```
```

Если загрузка оборвалась, просто повторите:

```
```powershell
```

```
ollama run qwen2.5-coder:7b
```

```
```
```

Ollama обычно продолжает или быстро проверяет уже скачанные части.

Обратите внимание: `qwen2.5-coder:7b` — это ****обычная модель, не abilitated****. Чтобы получить uncensored-вариант, необходимо отдельно иметь файл `.gguf`, например:

```
```text
```

```
Qwen2.5-Coder-7B-Instruct-abilitated.Q4_K_M.gguf
```

```
```
```

Проверьте содержимое вашей папки:

```
```powershell
```

```
Get-ChildItem H:\XPE\Models\qwen-uncensored
```

```
```
```

Если там есть `.gguf`, после завершения загрузки обычной модели можно будет импортировать uncensored-вариант через `Modelfile`. Если `.gguf` нет, одна только строка `Qwen2.5-Coder-7B-Instruct-abilitated` модель не скачает.

User prompt 9 of 25 - 9/8/2026, 10:28:39 PM:

PS H:\XPE> ollama run qwen2.5-coder:7b

pulling manifest

pulling 60e05f210007: 100%


 4.7 GB

Error: Head "https://registry.ollama.ai/v2/library/qwen2.5-coder/blobs/sha256:66b9ea09bd5b7099cbb4fc820f31b575c0366fa439b08245566692c6784e281e": net/http: TLS handshake timeout

PS H:\XPE> ollama list

| NAME | ID | SIZE | MODIFIED |
|------|----|------|----------|
|------|----|------|----------|

PS H:\XPE>

GPT-5.6 Luna:

<!-- **Troubleshooting download issues**

I need to figure out the download timeout problem. It might be a partial blob followed by a metadata request. I could retry and also check the network using `ollama pull`. If the model isn't installed until all layers are complete, I wonder if I need to set the environment or adjust the Ollama proxy. There's no known timeout setting, but maybe I could restart the service and check the firewall/VPN. I should make the answer concise. -->

Загрузка почти завершилась, но Ollama не смогла получить последний слой из-за тайм-аута TLS:

```text

TLS handshake timeout

```

Поэтому модель не добавилась в список:

```
```text
```

```
NAME ID SIZE
```

```
...
```

Это не ошибка видеокарты или памяти. Попробуйте повторить загрузку:

```
```powershell
```

```
ollama pull qwen2.5-coder:7b
```

```
...
```

После успешного завершения:

```
```powershell
```

```
ollama list
```

```
...
```

должна появиться строка `qwen2.5-coder:7b`.

Если снова будет тайм-аут:

1. Закройте Ollama из системного трея.
2. Перезапустите PowerShell от имени администратора.
3. Выполните:

```
```powershell
```

```
Stop-Process -Name ollama -Force -ErrorAction SilentlyContinue
```

```
ollama pull qwen2.5-coder:7b
```

```
...
```

Если используется VPN, прокси или нестабильный Wi-Fi, временно отключите VPN/прокси либо попробуйте другую сеть. Также проверьте, что на диске достаточно места:

```
```powershell
```

```
Get-PSDrive H
```

```
...
```

Для этой модели желательно иметь минимум **\*\*8–10 ГБ свободного места\*\***, потому что размер загрузки около 4,7 ГБ, а Ollama может временно хранить незавершённые слои.

После установки обычной модели запуск будет таким:

```
```powershell
```

```
ollama run qwen2.5-coder:7b
```

```
...
```

Но это всё ещё ****не uncensored-модель****. Если ваша цель именно abtiterated-версия, обычную `qwen2.5-coder:7b` можно не докачивать — потребуется отдельный `.gguf`-файл и импорт через `Modelfile`. Сейчас ошибка лишь означает, что текущая загрузка не завершилась.

User prompt 10 of 25 - 9/8/2026, 10:29:14 PM:

```
PS H:\XPE> ollama pull qwen2.5-coder:7b
```

```
pulling manifest
```

pulling 60e05f210007: 100%



4.7 GB

verifying sha256 digest

writing manifest

success

PS H:\XPE>

GPT-5.6 Luna:

Модель установилась успешно:

```text

success

```

Теперь можете запустить её:

```powershell

ollama run qwen2.5-coder:7b

```

Но это обычная модель Qwen2.5-Coder, а не uncensored-сборка. Проверить установку можно:

```powershell

ollama list

```

Запущенную модель и использование GPU/CPU:

```
```powershell
```

```
ollama ps
```

```
```
```

В новом окне PowerShell можно посмотреть нагрузку GTX 1050 Ti:

```
```powershell
```

```
nvidia-smi
```

```
```
```

Пока скачанная модель вам нужна только для проверки Ollama. Для вашей основной цели — uncensored-модели — выполните:

```
```powershell
```

```
Get-ChildItem H:\XPE\Models\qwen-uncensored
```

```
```
```

Если там есть файл `.gguf`, импортируйте его через `Modelfile`. Если папка пустая, uncensored-модель ещё не скачана.

User prompt 11 of 25 - 9/8/2026, 10:32:13 PM:

```
PS H:\XPE> ollama run qwen2.5-coder:7b
```

```
:
```

```
PS H:\XPE> ollama run qwen2.5-coder:7b
```

```
>>> Send a message (/? for help)
```

GPT-5.6 Luna:

Модель запустилась успешно. Вы уже находитесь в её интерактивном чате:

```
```text
```

```
>>> Send a message (/? for help)
```

```
```
```

Теперь вставьте, например:

```
```text
```

Ты — точный программист для разработки игр.

Работай только с кодом, который я присылаю.

Не переписывай проект целиком.

Не меняй имена переменных без разрешения.

Сначала укажи причину ошибки, затем покажи минимальное исправление.

Не добавляй библиотеки без моего разрешения.

```
```
```

Затем нажмите Enter и отправьте задачу, например:

```
```text
```

Напиши движение игрока для Godot 4 на GDScript.

Условия:

- вид сверху;

- управление WASD;

- скорость хранится в переменной speed;

- используется CharacterBody2D;

- выведи полный код скрипта;

- не используй сторонние библиотеки.

```
```
```

Выйти из чата можно сочетанием:

```
```text
```

```
Ctrl+C
```

```
```
```

Полезные команды внутри чата:

```
```text
```

```
/help
```

```
```
```

или:

```
```text
```

```
/?
```

```
```
```

Проверить загрузку модели на видеокарту:

```
```powershell
```

```
ollama ps
```

```
```
```

Если в столбце `PROCESSOR` будет указано сочетание CPU/GPU — это нормально для GTX 1050 Ti. Но запущенная модель сейчас всё ещё обычная `qwen2.5-coder:7b`, не abilitated/uncensored.

User prompt 12 of 25 - 9/8/2026, 10:32:48 PM:

PS H:\XPE> ollama run qwen2.5-coder:7b

>>> hi

Hello! How can I assist you today?

>>> Send a message (/? for help)

GPT-5.6 Luna:

Всё работает: модель отвечает корректно.

Сейчас запущена обычная модель:

```text

qwen2.5-coder:7b

```

Проверьте её работу на программировании:

```text

Напиши для Godot 4 скрипт CharacterBody2D на GDScript.

Нужны:

- движение WASD;
- переменная speed;
- нормализация диагонального движения;
- без сторонних библиотек;
- выведи только полный скрипт.

```

Чтобы завершить чат:

```
```text
```

```
Ctrl+C
```

```
```
```

Чтобы посмотреть, используется ли GTX 1050 Ti:

```
```powershell
```

```
ollama ps
```

```
```
```

Пока вы работаете именно с этой моделью. Для uncensored-варианта необходимо отдельно скачать `.gguf`-файл и создать на его основе модель через `Modelfile`; строка `Qwen2.5-Coder-7B-Instruct-abliterated` сама по себе командой не является.

User prompt 13 of 25 - 9/8/2026, 10:34:06 PM:

>>> Напиши для Godot 4 скрипт CharacterBody2D на GDScript.

... Нужны:

... - движение WASD;

... - переменная speed;

... - нормализация диагонального движения;

... - без сторонних библиотек;

... - выведи только полный скрипт.

...

Конечно! Вот полный скрипт на GDScript для `CharacterBody2D` в Godot 4, который реализует движение WASD с учетом

нормализации диагонального движения:

```
``gdscript

extends CharacterBody2D

# Скорость передвижения
export var speed = 200

# Вектор движения
var direction = Vector2.ZERO

func _ready():
    pass

func _process(delta):
    # Очистка вектора движения
    direction = Vector2.ZERO

    # Проверка нажатых клавиш
    if Input.is_action_pressed("ui_up"):
        direction.y -= 1
    if Input.is_action_pressed("ui_down"):
        direction.y += 1
    if Input.is_action_pressed("ui_left"):
        direction.x -= 1
    if Input.is_action_pressed("ui_right"):
        direction.x += 1

    # Нормализация вектора движения
    if direction.length() > 0:
```

```

direction = direction.normalized()

# Применение вектора движения
move_and_slide(direction * speed)
...

```

Этот скрипт делает следующее:

1. Определяет переменную `speed` для скорости передвижения.
2. Использует `direction` для хранения вектора движения.
3. В `\_process(delta)` проверяет нажатые клавиши WASD и соответствующим образом изменяет вектор движения.
4. Нормализует вектор движения, чтобы диагональное движение не ускоряло персонажа.
5. Применяет вектор движения к персонажу с помощью `move\_and\_slide`.

Не забудьте настроить кнопки управления в Godot через `Project -> Project Settings -> Input Map`.

>>> Send a message (/? for help)

GPT-5.6 Luna:

<!-- **Addressing Godot syntax issues**

I need to point out that the model provided the wrong Godot 3 syntax and incorrect WASD actions for the `move\_and\_slide` API. The user is likely showing output and seeking feedback. I should explain the model's accuracy and offer a corrected prompt or script. It's important to mention `@export`, `\_physics\_process`, and the velocity property. In Godot 4, `move\_and\_slide()` doesn't take arguments. I should recommend using `Input.get\_vector("move\_left"...)` for proper action configurations. -->

Модель работает, но сгенерировала код с ошибками для Godot 4:

- `export var speed` — синтаксис Godot 3;

- `\_process()` для физического движения использовать не следует;
- `move\_and\_slide(direction \* speed)` — неверно для Godot 4;
- `CharacterBody2D` использует встроенную переменную `velocity`;
- `ui\_up` и другие действия — это стрелки, а не обязательно WASD.

Корректный вариант для Godot 4:

```
``gdscript
```

```
extends CharacterBody2D
```

```
@export var speed: float = 200.0
```

```
func _physics_process(_delta: float) -> void:
```

```
    var direction := Input.get_vector(
```

```
        "move_left",
```

```
        "move_right",
```

```
        "move_up",
```

```
        "move_down"
```

```
    )
```

```
    velocity = direction * speed
```

```
    move_and_slide()
```

```
``
```

В `Project Settings` → `Input Map` нужно создать действия:

```
``text
```

```
move_left — A
```

```
move_right — D
```

move_up — W

move_down — S

...

`Input.get\_vector()` автоматически нормализует диагональное направление, поэтому персонаж не будет двигаться по диагонали быстрее.

Чтобы модель чаще выдавала правильный Godot 4, добавляйте к запросу:

```text

Используй строго Godot 4.x.

Проверь синтаксис Godot 4 и API CharacterBody2D.

Для движения используй velocity и move\_and\_slide() без аргументов.

Используй \_physics\_process(), а не \_process().

Перед ответом проверь код на несовместимость с Godot 3.

Выведи только полный скрипт без объяснения.

...

Для вашей модели лучше явно указывать версию движка и требовать проверку API: локальная 7B-модель неплохо пишет простые скрипты, но часто смешивает Godot 3 и Godot 4.

-----

User prompt 14 of 25 - 9/8/2026, 10:34:35 PM:

z gjghjcbk yt wtypehbhjdfyye. gjvybim,

GPT-5.6 Luna:

<!-- -->

Понял: **\*\*вы просили нецензурированную модель\*\***, а я направил вас на обычную ``qwen2.5-coder:7b``. Исправляю.

Установленная модель:

```
```text
```

```
qwen2.5-coder:7b
```

```
```
```

не является uncensored. Системная инструкция не убирает встроенные ограничения — нужна отдельная abilitated/uncensored-сборка в формате ``gguf``.

Сейчас выйдите из Ollama:

```
```text
```

```
Ctrl+C
```

```
```
```

Проверьте, есть ли уже файл модели в вашей папке:

```
```powershell
```

```
Get-ChildItem H:\XPE\Models\qwen-uncensored -Recurse
```

```
```
```

Если увидите файл с расширением ``gguf``, пришлите его точное имя. Если папка пустая, нужно скачать **\*\*Qwen2.5-Coder-7B-Instruct-abilitated Q4\_K\_M GGUF\*\***, затем импортировать его в Ollama.

После появления файла ``gguf`` команды будут такими:

```
```powershell
```

```
cd H:\ХРЕ\Models\qwen-uncensored
```

```
notepad .\Modelfile
```

```
```
```

В `Modelfile`:

```
```text
```

```
FROM ./ИМЯ_ВАШЕГО_ФАЙЛА.gguf
```

```
PARAMETER num_ctx 4096
```

```
PARAMETER temperature 0.2
```

```
PARAMETER top_p 0.9
```

```
```
```

Создание и запуск:

```
```powershell
```

```
ollama create qwen-coder-uncensored -f .\Modelfile
```

```
ollama run qwen-coder-uncensored
```

```
```
```

Не вводите название `Qwen2.5-Coder-7B-Instruct-abliterated` напрямую в PowerShell — это название файла или сборки, а не команда.

-----

User prompt 15 of 25 - 9/8/2026, 10:35:16 PM:

```
PS H:\ХРЕ> Get-ChildItem H:\ХРЕ\Models\qwen-uncensored -Recurse
```

PS H:\ХРЕ>

GPT-5.6 Luna:

Папка пустая — uncensored-модель ещё не скачана. Установленная `qwen2.5-coder:7b` — обычная версия.

Сейчас лучше не скачивать случайные файлы из непроверенных источников. Для вашей системы ищите именно:

```text

Qwen2.5-Coder-7B-Instruct abliterated GGUF Q4_K_M

```

Нужен файл с расширением:

```text

.gguf

```

и размером примерно **\*\*4–5 ГБ\*\***. Выбирайте `Q4\_K\_M`; `Q8` для вашей GTX 1050 Ti будет слишком тяжёлой.

После скачивания переместите файл в:

```text

H:\ХРЕ\Models\qwen-uncensored

```

Проверьте:

```
```powershell
```

```
Get-ChildItem H:\XPE\Models\qwen-uncensored
```

```
```
```

Например, если файл называется:

```
```text
```

```
qwen2.5-coder-7b-instruct-abliterated.Q4_K_M.gguf
```

```
```
```

создайте `Modelfile`:

```
```powershell
```

```
cd H:\XPE\Models\qwen-uncensored
```

```
notepad .\Modelfile
```

```
```
```

Вставьте:

```
```text
```

```
FROM ./qwen2.5-coder-7b-instruct-abliterated.Q4_K_M.gguf
```

```
PARAMETER num_ctx 4096
```

```
PARAMETER temperature 0.2
```

```
PARAMETER top_p 0.9
```

```
PARAMETER repeat_penalty 1.1
```

```
SYSTEM ""
```

Ты — точный помощник-программист.

Отвечай непосредственно по задаче.

Не добавляй ненужные отказы, морализаторство и отвлечённые рассуждения.

Не переписывай проект целиком без запроса.

Не меняй имена переменных без разрешения.

Не выдумывай API.

""""

'''

Затем:

```powershell

ollama create qwen-coder-uncensored -f .\Modelfile

ollama run qwen-coder-uncensored

'''

Проверка:

```powershell

ollama list

'''

Важно: модель с пометкой `abliterated` обычно означает ослабленное поведение отказов, но это не абсолютная гарантия полного отсутствия ограничений. Для вашей задачи разработки игры такая сборка будет ближе к тому, что вы просите, чем установленная сейчас обычная модель.

User prompt 16 of 25 - 9/8/2026, 10:36:00 PM:

слушай я же тебя попросил найти ее

GPT-5.6 Luna:

<!-- **Searching for model download**

I realize I need to apologize and perform a web search right now for the current model download. It seems I should look for actual downloads using specific terms related to it. The user asked me to find it, and I'm going to prioritize that search. I'm focusing on finding the right information to meet their needs, so I'll make sure to search for the exact terms required. -->

Да, вы правы — нужно было сразу найти и дать конкретную модель. Подходящая сборка нашлась:

Qwen2.5-Coder-7B-Instruct-abliterated-GGUF от bartowski. Она помечена как `abliterated`/`uncensored`, а нужный файл — **Q4_K_M**, размер около **4,68 ГБ**. [1,2]

Откройте страницу модели:

`https://huggingface.co/bartowski/Qwen2.5-Coder-7B-Instruct-abliterated-GGUF`

В разделе файлов скачайте:

```text

Qwen2.5-Coder-7B-Instruct-abliterated-Q4\_K\_M.gguf

```

Сохраните его в:

```text

H:\XPE\Models\qwen-uncensored

```

После скачивания выполните:

```
```powershell
cd H:\ХРЕ\Models\qwen-uncensored
```
```

Создайте `Modelfile`:

```
```powershell
notepad .\Modelfile
```
```

Вставьте:

```
```text
FROM ./Qwen2.5-Coder-7B-Instruct-abliterated-Q4_K_M.gguf

PARAMETER num_ctx 4096
PARAMETER temperature 0.2
PARAMETER top_p 0.9
PARAMETER repeat_penalty 1.1
```
```

Сохраните файл и создайте модель в Ollama:

```
```powershell
ollama create qwen-coder-uncensored -f .\Modelfile
```
```

Запустите:

```
```powershell
ollama run qwen-coder-uncensored
```
```

Проверьте, что она появилась:

```
```powershell
ollama list
```
```

Для GTX 1050 Ti лучше начинать с `num\_ctx 4096`. Если будет медленно или появятся ошибки нехватки памяти, замените в `Modelfile`:

```
```text
PARAMETER num_ctx 2048
```
```

и снова выполните:

```
```powershell
ollama create qwen-coder-uncensored -f .\Modelfile
```
```

Эта модель не находится через `ollama run` напрямую, потому что это не опубликованный тег библиотеки Ollama, а GGUF-файл с Hugging Face, который нужно импортировать вручную.

[Read More](#)

[1] bartowski/Qwen2.5-Coder-7B-Instruct-abliterated-GGUF —
<https://huggingface.co/bartowski/Qwen2.5-Coder-7B-Instruct-abliterated-GGUF>

[2] Qwen2.5-Coder-7B-Instruct-abliterated-Q4_K_M.gguf —
https://huggingface.co/bartowski/Qwen2.5-Coder-7B-Instruct-abliterated-GGUF/blob/d8d6a8862d1960ab40372036a7cb49e46e3523ef/Qwen2.5-Coder-7B-Instruct-abliterated-Q4_K_M.gguf

User prompt 17 of 25 - 9/8/2026, 10:38:15 PM:

пока я скачиваю нужно сделать статичный TUI для POWERSHELL 7 для того чтобы было легче все отличать по стандартам визуала агентов

GPT-5.6 Luna:

<!-- **Considering static TUI options**

I'm thinking about creating a static TUI in PowerShell 7 to help distinguish agents visually. This would involve writing a script that includes panels, colors, statuses, logs, and agent labels. I need to clarify the requirements for the domain before moving forward. I should ensure that the PowerShell script is compatible with PS7 and doesn't rely on external modules. The goal could be to design a dashboard that effectively displays agent statuses while keeping it concise and informative. -->

Сделаем статичный TUI без сторонних модулей: верхняя панель проекта, список агентов слева, текущий агент по центру, журнал событий справа/снизу. Он будет работать в PowerShell 7 через обычные ANSI escape-последовательности.

Создайте файл:

```
```powershell
notepad H:\XPE\agent-ui.ps1
```
```

Вставьте код:

```
```powershell
```

```
#requires -Version 7.0
```

```
$esc = [char]27
```

```
$reset = "$esc[0m"
```

```
$colors = @{
```

```
 Header = "$esc[38;5;39m"
```

```
 Cyan = "$esc[38;5;51m"
```

```
 Green = "$esc[38;5;82m"
```

```
 Yellow = "$esc[38;5;220m"
```

```
 Red = "$esc[38;5;203m"
```

```
 Gray = "$esc[38;5;245m"
```

```
 White = "$esc[38;5;255m"
```

```
 BlueBg = "$esc[48;5;24m"
```

```
 DarkBg = "$esc[48;5;235m"
```

```
}
```

```
$agents = @(
```

```
 [pscustomobject]@{
```

```
 Name = "ARCHITECT"
```

```
 Role = "Архитектура проекта"
```

```
 Status = "READY"
```

```
 Color = $colors.Cyan
```

```
 },
```

```
 [pscustomobject]@{
```

```
 Name = "CODER"
```

```
 Role = "Написание кода"
```

```

 Status = "ACTIVE"

 Color = $colors.Green
 },
 [pscustomobject]@{
 Name = "DEBUGGER"
 Role = "Поиск ошибок"
 Status = "IDLE"
 Color = $colors.Yellow
 },
 [pscustomobject]@{
 Name = "REVIEWER"
 Role = "Проверка изменений"
 Status = "IDLE"
 Color = $colors.Gray
 }
}
)

```

```

$logs = [System.Collections.Generic.List[string]]::new()
$logs.Add("Система визуальных агентов запущена")
$logs.Add("Проект: XPE")
$logs.Add("Движок: Godot 4")
$logs.Add("Модель: ожидает загрузки")
$logs.Add("Режим: локальный")

```

```

function Write-At {
 param(
 [int]$X,
 [int]$Y,
 [string]$Text,
)
}

```

```

 [string]$Color = $reset
)

 Write-Host "$esc[$($Y);$($X)H$Color$Text$reset" -NoNewline
}

function Get-BoxLine {
 param(
 [int]$Width,
 [string]$Left = "|",
 [string]$Right = "|"
)

 return "$Left" + ("—" * ($Width - 2)) + "$Right"
}

function Draw-Box {
 param(
 [int]$X,
 [int]$Y,
 [int]$Width,
 [int]$Height,
 [string]$Title,
 [string]$Color = $colors.Header
)

 Write-At $X $Y ("└─ $Title " + ("—" * [Math]::Max(0, $Width - $Title.Length - 5)) + "┐") $Color

 for ($i = 1; $i -lt ($Height - 1); $i++) {

```

```

 Write-At $X ($Y + $i) (Get-BoxLine -Width $Width) $colors.Gray
 }

 Write-At $X ($Y + $Height - 1) ("└" + ("─" * ($Width - 2)) + "┘") $colors.Gray
}

function Draw-UI {
 Clear-Host

 $width = $Host.UI.RawUI.WindowSize.Width
 $height = $Host.UI.RawUI.WindowSize.Height

 if ($width -lt 100 -or $height -lt 28) {
 Write-Host "Увеличьте окно PowerShell минимум до 100x28." -ForegroundColor Yellow
 return
 }

 $leftWidth = 27
 $rightWidth = $width - $leftWidth - 5
 $mainHeight = $height - 10

 # Заголовок
 Write-At 1 1 ("═" * $width) $colors.Header
 Write-At 3 2 "XPE // AGENT CONTROL CENTER" $colors.Cyan
 Write-At ($width - 30) 2 "POWERSHELL 7 // LOCAL" $colors.Gray
 Write-At 1 3 ("═" * $width) $colors.Header

 # Левая панель
 Draw-Box 2 5 $leftWidth $mainHeight "AGENTS" $colors.Cyan

```

```

$row = 7

foreach ($agent in $agents) {
 $statusColor = switch ($agent.Status) {
 "ACTIVE" { $colors.Green }
 "READY" { $colors.Cyan }
 "ERROR" { $colors.Red }
 default { $colors.Gray }
 }

 Write-At 4 $row "●" $statusColor
 Write-At 7 $row $agent.Name $agent.Color
 Write-At 7 ($row + 1) $agent.Role $colors.Gray
 Write-At 7 ($row + 2) $agent.Status $statusColor

 $row += 5
}

```

# Центральная панель

```
Draw-Box ($leftWidth + 4) 5 $rightWidth $mainHeight "CURRENT TASK" $colors.Green
```

```
Write-At ($leftWidth + 7) 7 "PROJECT" $colors.Gray
```

```
Write-At ($leftWidth + 17) 7 "XPE / GAME" $colors.White
```

```
Write-At ($leftWidth + 7) 9 "ENGINE" $colors.Gray
```

```
Write-At ($leftWidth + 17) 9 "Godot 4" $colors.White
```

```
Write-At ($leftWidth + 7) 11 "MODEL" $colors.Gray
```

```
Write-At ($leftWidth + 17) 11 "Qwen2.5-Coder 7B" $colors.White
```

```
Write-At ($leftWidth + 7) 13 "ACTIVE AGENT" $colors.Gray
```

```
Write-At ($leftWidth + 17) 13 "CODER" $colors.Green
```

```
Write-At ($leftWidth + 7) 16 "TASK" $colors.Gray
```

```
Write-At ($leftWidth + 17) 16 "Ожидание задания..." $colors.Yellow
```

```
Журнал
```

```
$logY = $height - 7
```

```
Draw-Box 2 $logY ($width - 4) 5 "EVENT LOG" $colors.Yellow
```

```
$visibleLogs = $logs | Select-Object -Last 3
```

```
$line = $logY + 1
```

```
foreach ($log in $visibleLogs) {
```

```
 $text = " " + $log
```

```
 if ($text.Length -gt ($width - 8)) {
```

```
 $text = $text.Substring(0, $width - 11) + "..."
```

```
 }
```

```
 Write-At 4 $line $text $colors.Gray
```

```
 $line++
```

```
}
```

```
Нижняя строка
```

```
Write-At 3 ($height - 1) "[R] refresh [L] add log [Q] quit" $colors.Gray
```

```
}
```

```
[Console]::CursorVisible = $false
```

```
try {
 while ($true) {
 Draw-UI

 if ([Console]::KeyAvailable) {
 $key = [Console]::ReadKey($true).Key

 switch ($key) {
 "R" {
 $logs.Add("Интерфейс обновлён")
 }

 "L" {
 $logs.Add("Тестовое событие добавлено")
 }

 "Q" {
 break
 }
 }
 }

 Start-Sleep -Milliseconds 150
 }
}
finally {
 [Console]::CursorVisible = $true
 Clear-Host
```

```
Write-Host "Agent UI завершён." -ForegroundColor Cyan
}
...

```

Запуск:

```
``powershell
pwsh -ExecutionPolicy Bypass -File H:\XPE\agent-ui.ps1
...

```

Или из PowerShell 7:

```
``powershell
H:\XPE\agent-ui.ps1
...

```

Управление:

```
``text
R — обновить интерфейс
L — добавить тестовое событие в журнал
Q — выйти
...

```

Визуальный стандарт здесь такой:

```
``text
CYAN — информация и готовность
GREEN — активный агент или успешное состояние

```

YELLOW — ожидание, предупреждение, задача

RED — ошибка

GRAY — неактивный элемент

...

Следующим шагом можно подключить реальные данные:

- статус Ollama через `ollama ps`;
- активную модель;
- загрузку CPU/RAM/GPU;
- список файлов проекта;
- отдельные окна для `ARCHITECT`, `CODER`, `DEBUGGER` и `REVIEWER`;
- журнал команд и результатов модели.

-----

User prompt 18 of 25 - 9/8/2026, 10:50:34 PM:

он мигает прямо пиздец а еще выглядит как говно следующим сообщением скину  
примеры так что можешь не отвечать

[Attached image 1]

GPT-5.6 Luna:

Понял. Жду примеры.

-----

User prompt 19 of 25 - 9/8/2026, 10:54:53 PM:

вот картинка Cursor и файл Google ai вот еще примеры

[Attached image 1][Attached image 2][Attached image 3][Attached file chat.pdf][Attached file  
chat2.pdf]

GPT-5.6 Luna:

```
<!-- **Summarizing implementation plan**
```

I need to acknowledge the design target, noting they haven't provided code yet. They shared examples, specifying they want an interactive terminal app instead of a static dashboard, with details like no flicker, dark panes, borders, input, chat history, and agent tabs. I'll mention potential limitations of PowerShell and suggest using Textual. But PowerShell 7 does allow .NET console raw rendering or PSReadLine. I'll confirm I got it and plan to build a modern TUI based on their examples. -->

Понял направление. Нужен не «дашборд с рамками», а настоящий интерактивный TUI в стиле Cursor / OpenRouter / K9:

- тёмный минималистичный интерфейс;
- левая боковая панель с агентами, сессиями и проектами;
- центральная область чата/терминала;
- отдельные панели для вывода инструментов и статуса;
- поле ввода внизу;
- переключение фокуса клавишей `Tab`;
- выбор агента стрелками;
- команды вроде `/new`, `/model`, `/agents`, `/clear`;
- подсветка ролей, команд, ошибок и результатов;
- кодовые блоки с моноширинным оформлением;
- стабильный экран без постоянного `Clear-Host` и мерцания.

Предыдущий вариант нужно выбросить полностью. Для PowerShell 7 сделаем перерисовку только изменившихся областей через ANSI-курсор, а не обновление всего экрана. Основа будет примерно такая:

```
```text
```

XPE AGENT WORKSPACE

PROJECTS	SESSION: GODOT GAME	STATUS
----------	---------------------	--------

блин возьми за пример хотябы жто

[Attached image 1]

GPT-5.6 Luna:

Да, теперь понял. Визуальный ориентир — именно ****K9/Kinic****:

- почти чёрный фон;
- тонкие розово-фиолетовые границы;
- крупный ASCII-логотип сверху;
- узкая левая панель;
- вкладки под логотипом;
- большая центральная область чата;
- аккуратное поле ввода снизу;
- небольшая статусная строка;
- никакой постоянной перерисовки и мигания;
- минимальное количество цветов;
- интерфейс должен выглядеть как единое приложение, а не как набор PowerShell-таблиц.

Сделаем структуру:

```
```text
```

K9/XPE LOGO

• 1 item

◆ model: local

Tabs

Chat [Agent / Session]

Search

You:

сообщение

```

| Items | | | |
| | Agent: |
| ● CODER | ответ модели |
| ○ DEBUGGER | |
| | |
| | ┌──────────┐ |
| | | > Введите сообщение... | |
| | └──────────┘ |
| ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐
| | Shift+Enter send Esc close /new Tab focus |
| └──────────┘ └──────────┘ └──────────┘ └──────────┘
...

```

Розовый цвет будет главным акцентом:

```text

границы: #D9469F

активный: #F472B6

текст: #D4D4D8

вторичный: #71717A

фон: #18181B

панели: #202023

```

Предыдущий вариант с голубыми рамками, множеством боксов и постоянным `Clear-Host` больше не используем. Нужен будет **\*\*экранный буфер\*\***, который рисует интерфейс один раз и меняет только конкретную область чата или ввода. Это уберёт мерцание.

Для PowerShell 7 я сделаю это не на чистых `Write-Host`-перерисовках, а через:

- ANSI escape sequences;
- позиционирование курсора;
- альтернативный экран терминала;
- отдельный буфер текста;
- обработку клавиш без очистки всего окна;
- фиксированные области интерфейса.

Внешне ориентируемся именно на этот пример Kinis, но адаптируем под XPE: слева агенты и сессии, по центру чат с Ollama, сверху модель и проект, снизу управление.

-----

User prompt 21 of 25 - 9/8/2026, 10:58:31 PM:

я хочу еще сделать мост к gimiini чтобы я мог говорить локальному ии тип у нас есть nodejs есть playwright Typescript и он может видеть вкладку с gimiini и писать он может и gimiini может отвечать пусть говорят распределяют задачи а я если чтото то вставлю свои пять копеек

GPT-5.6 Luna:

<!-- -->

Да, это можно сделать. Получится **\*\*мультиагентный мост\*\***:

```text

Ты

|

▼

XPE TUI

|

▼

Node.js-оркестратор

- └— Ollama / локальный кодер
- └— Playwright → вкладка Gemini
- └— файловые инструменты
- └— Git / тесты / Godot
- └— журнал сообщений
- ...

Локальный ИИ и Gemini смогут обмениваться сообщениями через центральный оркестратор:

```text

Локальный ИИ:

"Задача требует анализа архитектуры. Передаю Gemini."

Gemini:

"Предлагаю структуру PlayerController и систему состояний."

Локальный ИИ:

"Принимаю вариант. Реализую код и проверю ошибки."

Ты:

"Изменить управление с WASD на схему из Input Map."

...

Главное — не давать моделям напрямую бесконтрольно менять проект. Лучше разделить роли:

| Агент       | Роль                   | Доступ                  |
|-------------|------------------------|-------------------------|
| Local Coder | пишет и исправляет код | чтение/изменение файлов |

| Gemini Analyst | архитектура, идеи, ревью | только проектный контекст |  
| Debugger | анализ ошибок | чтение логов и тесты |  
| Human | утверждение решений | финальное управление |

Для начала структура Node.js-проекта:

```
```text
xpe/
├─ src/
|   ├─ main.ts
|   ├─ orchestrator.ts
|   ├─ ollama.ts
|   ├─ gemini-browser.ts
|   ├─ messages.ts
|   └─ tools.ts
├─ ui/
|   └─ tui.ts
├─ workspace/
|   ├─ inbox/
|   ├─ outbox/
|   └─ approvals/
├─ package.json
└─ tsconfig.json
```
```

Установка:

```
```powershell
mkdir H:\XPE\bridge
```

```
cd H:\XPE\bridge
```

```
npm init -y
```

```
npm install playwright
```

```
npm install -D typescript tsx @types/node
```

```
npx playwright install chromium
```

```
...
```

Для локальной модели можно обращаться к Ollama через HTTP API:

```
```typescript
```

```
// src/ollama.ts
```

```
export async function askLocal(prompt: string): Promise<string> {
```

```
 const response = await fetch("http://127.0.0.1:11434/api/generate", {
```

```
 method: "POST",
```

```
 headers: {
```

```
 "Content-Type": "application/json"
```

```
 },
```

```
 body: JSON.stringify({
```

```
 model: "qwen2.5-coder:7b",
```

```
 prompt,
```

```
 stream: false,
```

```
 options: {
```

```
 temperature: 0.2,
```

```
 num_ctx: 4096
```

```
 }
 })
```

```
 });
```

```

if (!response.ok) {
 throw new Error(`Ollama HTTP ${response.status}`);
}

const data = await response.json() as { response: string };
return data.response;
}
...

```

Мост к уже открытой вкладке Gemini через Playwright лучше делать через отдельный профиль Chromium. Playwright подключается к браузеру с включённым remote debugging:

```

```powershell
$chrome = "${env:ProgramFiles(x86)}\Google\Chrome\Application\chrome.exe"

& $chrome `
  --remote-debugging-port=9222 `
  --user-data-dir="H:\XPE\chrome-profile"
...

```

В этом окне один раз откройте Gemini и войдите в аккаунт. Затем TypeScript сможет подключиться к вкладке:

```

```typescript
// src/gemini-browser.ts

import { chromium, Page } from "playwright";

export async function connectGemini(): Promise<Page> {
 const browser = await chromium.connectOverCDP(
 "http://127.0.0.1:9222"
);
}

```

```
);
```

```
for (const context of browser.contexts()) {
 for (const page of context.pages()) {
 if (page.url().includes("gemini.google.com")) {
 await page.bringToFront();
 return page;
 }
 }
}
```

```
throw new Error("Вкладка Gemini не найдена");
}
```

```
export async function sendToGemini(
 page: Page,
 message: string
): Promise<void> {
 const input = page.locator(
 'textarea, [contenteditable="true"]'
).last();

 await input.waitFor({ state: "visible", timeout: 15_000 });
 await input.fill(message);
 await input.press("Enter");
}
```

```
export async function readGeminiAnswer(
 page: Page
```

```

): Promise<string> {
 await page.waitForTimeout(1500);

 const candidates = page.locator(
 '[data-message-author-role="model"], model-response, message-content'
);

 const count = await candidates.count();

 if (count === 0) {
 return "Ответ Gemini не найден.";
 }

 return (await candidates.last().innerText()).trim();
}
...

```

Селекторы Gemini могут меняться, поэтому их лучше хранить отдельно в конфигурации:

```

``typescript
export const selectors = {
 input: [
 "textarea",
 '[contenteditable="true"]'
],
 responses: [
 '[data-message-author-role="model"]',
 "model-response",
 "message-content"
]
}

```

```
]
};
...

```

Центральный оркестратор должен передавать сообщения в едином формате:

```
```typescript
export type AgentName = "local" | "gemini" | "human";

export type AgentMessage = {
  id: string;
  from: AgentName;
  to: AgentName | "broadcast";
  type: "task" | "proposal" | "review" | "result" | "question";
  content: string;
  files?: string[];
  requiresApproval?: boolean;
  timestamp: string;
};
...

```

Пример распределения задачи:

```
```typescript
const task = `
Проект Godot 4.

Нужно создать систему движения игрока.

Ограничения:
- CharacterBody2D;

```

- Input Map;
- WASD;
- без сторонних библиотек;
- код должен быть совместим с Godot 4.

`;

```
const geminiPrompt = `
```

Ты архитектор проекта.

Проанализируй задачу и предложи структуру решения.

Не пиши большой объём кода.

Укажи риски и API Godot 4.

ЗАДАЧА:

\${task}

`;

```
const architecture = await sendToGeminiAndRead(geminiPrompt);
```

```
const localPrompt = `
```

Ты исполнительный программист.

На основе архитектурного предложения создай точное решение.

Покажи изменяемые файлы и минимальный патч.

Не меняй код без необходимости.

ЗАДАЧА:

\${task}

ПРЕДЛОЖЕНИЕ АРХИТЕКТОРА:

\${architecture}

`;

```
const implementation = await askLocal(localPrompt);
```

...

Лучший режим для вас — **\*\*полуавтоматический\*\***:

```text

1. Вы вводите задачу.
2. Gemini предлагает архитектуру.
3. Локальная модель проверяет предложение.
4. Вы видите диалог в TUI.
5. Вы подтверждаете.
6. Локальная модель меняет файлы.
7. Тесты запускаются автоматически.
8. Ошибка возвращается локальному ИИ и Gemini.

...

Добавьте обязательные команды подтверждения:

```text

|                     |                                   |
|---------------------|-----------------------------------|
| /ask gemini <текст> | отправить вопрос Gemini           |
| /ask local <текст>  | отправить вопрос локальной модели |
| /debate <текст>     | дать задачу обоим агентам         |
| /approve            | разрешить изменение файлов        |
| /reject             | отклонить предложение             |
| /diff               | показать изменения                |
| /test               | запустить тесты                   |
| /stop               | остановить текущую задачу         |

```

Особенно важен режим `requiresApproval`:

```
```typescript
if (message.requiresApproval) {
 console.log("Изменения ожидают подтверждения пользователя.");
 console.log("Введите /approve или /reject.");
}
```
```

Не стоит сразу строить полностью автономный цикл. Сначала сделайте минимальный рабочий прототип:

```
```text
TUI → Node.js → Ollama
TUI → Node.js → Playwright → Gemini
Node.js → общий журнал сообщений
```
```

Затем добавьте:

```
```text
локальный ИИ ↔ Gemini
```
```

И только после этого:

```
```text
```

агенты → файлы → Godot → тесты → исправление

...

Такой подход даст именно то, что вы описали: локальный ИИ будет исполнителем, Gemini — вторым архитектором/ревьюером, а вы останетесь оператором, который может вмешаться в любой момент.

-----

User prompt 22 of 25 - 9/8/2026, 10:59:51 PM:

подожди сначала tui сделаем мне не удобно сообщем еще эти моргания

GPT-5.6 Luna:

<!-- -->

Да, сначала полностью делаем **\*\*TUI\*\***, без Ollama, Gemini и агентов. Мост пока не трогаем.

Основная проблема текущей версии — она постоянно вызывает:

```powershell

Clear-Host

```

Из-за этого весь экран заново рисуется и мигает. В новой версии этого не будет.

Сначала сделаем только визуальный прототип в стиле Kinic:

- тёмный фон;
- розово-фиолетовая тема;
- крупный логотип;

- левая боковая панель;
- вкладки;
- центральная область чата;
- поле ввода;
- нижняя строка команд;
- статичные тестовые сообщения;
- навигация клавишами;
- без сетки из лишних рамок;
- без постоянного обновления экрана;
- без подключения моделей.

Технически:

```text

экран рисуется один раз

↓

курсор скрывается

↓

изменяются только нужные строки

↓

никакого Clear-Host

↓

никакого мерцания

```

Сделаем сначала такой каркас:

```text

```
|_____|
```

```

| XPE          • 1 item      |
| █████ логотип                |
♦ model: local
Tabs
Search
Items
• CODER
○ DEBUGGER
-----
Tab focus Enter send /new Esc close Q quit
-----
'''

```

Сначала добиваемся нормального визуального вида и отсутствия мигания. Только после вашего одобрения добавим ввод текста, переключение панелей и реальные данные.

User prompt 23 of 25 - 9/8/2026, 11:00:53 PM:

почему нельзя блин как в linux сделать красиво и со вкусом

GPT-5.6 Luna:

<!-- -->

Можно. Проблема не в PowerShell 7, а в том, что предыдущий вариант был сделан на примитивном `Write-Host` и ручных рамках. Это не настоящий TUI, поэтому он и выглядел плохо.

Для интерфейса уровня Linux-программ нужно использовать TUI-фреймворк, а не рисовать весь экран вручную. Оптимальный вариант для вас:

```
```text
```

PowerShell 7

↓ запускает

Node.js + TypeScript

↓

Ink / React или blessed

↓

настоящий TUI без мерцания

```
```
```

Но для сложного интерфейса с панелями, вкладками, фокусом, прокруткой и чатами лучше использовать **Python Textual** или **Go Bubble Tea**. На Windows они тоже работают в PowerShell 7 и выглядят как нормальные Linux TUI.

Я бы выбрал:

```
```text
```

Python + Textual

```
```
```

Почему:

- нормальные панели и виджеты;
- CSS-подобное оформление;

- фиксированные области;
- прокрутка;
- поля ввода;
- вкладки;
- фокус клавиатуры;
- отсутствие мерцания;
- простое создание интерфейса в стиле Kinic;
- легко подключить Ollama и Node.js позже.

Пример структуры:

```
```text
xpe/
├─ tui/
| ├─ app.py
| └─ styles.tcss
└─ widgets/
 └─ bridge/
 └─ Node.js
└─ workspace/
...
```
```

Установка:

```
```powershell
py -m pip install textual
...
```
```

Минимальный настоящий интерфейс будет выглядеть примерно так:

```

```python

from textual.app import App, ComposeResult

from textual.containers import Horizontal, Vertical

from textual.widgets import Header, Footer, Input, Label, ListItem, ListView, Static,
TabbedContent, TabPane

class XPE(App):

 CSS_PATH = "styles.tcss"

 def compose(self) -> ComposeResult:

 yield Header(show_clock=False)

 with Horizontal(id="layout"):

 with Vertical(id="sidebar"):

 yield Label("K9 / XPE", id="logo")

 yield Label("Tabs", classes="section")

 yield Label("Memories", classes="tab active")

 yield Label("Agents", classes="tab")

 yield Label("Sessions", classes="tab")

 yield Label("Search", classes="section")

 yield ListView(

 ListItem(Label("● CODER")),

 ListItem(Label("○ DEBUGGER")),

 ListItem(Label("○ REVIEWER")),

 id="agents"

)

```

```

with Vertical(id="main"):

 yield Label("◆ Chat [XPE SESSION]", id="title")

 yield Static(

 "You:\n"

 "Нужно создать систему движения игрока.\n\n"

 "Local Coder:\n"

 "Ожидаю задачу...",

 id="chat"

)

 yield Input(

 placeholder="Ask about this item... (Enter to send)",

 id="input"

)

```

```

yield Footer()

```

```

if __name__ == "__main__":
 XPE().run()
...

```

Стили:

```

```css

```

```

Screen {

    background: #18181b;

    color: #d4d4d8;

}

```

```

Header {

```

```
background: #18181b;

color: #f472b6;

dock: top;

}
```

```
Footer {

background: #18181b;

color: #71717a;

}
```

```
#layout {

height: 1fr;

}
```

```
#sidebar {

width: 28;

border: solid #d9469f;

padding: 1 2;

}
```

```
#main {

width: 1fr;

border: solid #d9469f;

padding: 1 2;

}
```

```
#logo {

color: #f472b6;

text-style: bold;
```

```
margin-bottom: 2;  
}
```

```
.section {  
  color: #a855f7;  
  margin-top: 1;  
}
```

```
.tab {  
  color: #a1a1aa;  
  padding: 0 1;  
}
```

```
.active {  
  color: #f472b6;  
  text-style: bold;  
}
```

```
#agents {  
  background: #18181b;  
  border: none;  
  height: auto;  
}
```

```
#title {  
  color: #f472b6;  
  text-style: bold;  
}
```

```
#chat {  
    height: 1fr;  
    border: solid #3f3f46;  
    padding: 1 2;  
    margin: 1 0;  
}
```

```
#input {  
    border: solid #d9469f;  
    background: #202023;  
}  
...
```

Запуск из PowerShell:

```
```powershell  
cd H:\XPE\tui
py .\app.py
...
```

Это уже будет ближе к Linux TUI, а не к мигающему PowerShell-скрипту.

Дальше можно сделать крупный ASCII-логотип `K9`, вкладки `Memories / Insert / Create / Market / Settings`, нормальную историю чата, навигацию `Tab`, горячие клавиши и подключение к Ollama. Сначала нужно перейти на нормальный TUI-фреймворк — вручную рисовать такой интерфейс через PowerShell не стоит.

-----

User prompt 24 of 25 - 9/8/2026, 11:02:14 PM:

```
PS C:\Users\RJ> cd H:\XPE\Models\qwen-uncensored
```

```
PS H:\XPE\Models\qwen-uncensored> cd ../../
```

```
PS H:\XPE> py -m pip install textual
```

Can't find a default Python.

```
PS H:\XPE> cd C:\Python314
```

```
PS C:\Python314> py -m pip install textual
```

Can't find a default Python.

```
PS C:\Python314>
```

GPT-5.6 Luna:

```
<!-- **Explaining Python launcher issues**
```

I need to explain the issue with the Python launcher being absent or not configured, which might relate to the Python executable itself. I should check the version using `python --version` or look for the command with `Get-Command python` and pip. It's possible that the Python314 folder exists without `python.exe`. In that case, I could use `C:\Python314\python.exe -m pip...`. Additionally, if pip is absent, I might need to run `-m ensurepip`. Lastly, the error "Can't find default Python" likely stems from no registered install. -->

`py` у вас установлен или вызывается, но не видит зарегистрированный Python. Переход в папку `C:\Python314` не меняет настройки запуска.

Сначала проверьте, есть ли сам интерпретатор:

```
```powershell
```

```
Test-Path C:\Python314\python.exe
```

```
```
```

Если результат:

```
```text
```

```
True
```

```
```
```

используйте прямой путь:

```
```powershell
```

```
C:\Python314\python.exe -m pip install textual
```

```
```
```

Если `pip` отсутствует:

```
```powershell
```

```
C:\Python314\python.exe -m ensurepip --upgrade
```

```
C:\Python314\python.exe -m pip install --upgrade pip
```

```
C:\Python314\python.exe -m pip install textual
```

```
```
```

Проверьте Python:

```
```powershell
```

```
C:\Python314\python.exe --version
```

```
```
```

Если получите ошибку, найдите настоящий путь:

```
```powershell
```

```
Get-Command python -ErrorAction SilentlyContinue
```

```
Get-Command python3 -ErrorAction SilentlyContinue
```

Get-Command py -ErrorAction SilentlyContinue

```

Также:

```powershell

where.exe python

where.exe py

```

Если Python установлен в другой папке, команда будет выглядеть так:

```powershell

"C:\ПУТЬ\К\python.exe" -m pip install textual

```

Можно проверить стандартные варианты:

```powershell

Get-ChildItem `

"\$env:LocalAppData\Programs\Python" `

"C:\Program Files\Python*" `

"C:\Python*" `

-Filter python.exe `

-Recurse `

-ErrorAction SilentlyContinue

```

После установки Textual запускать приложение нужно также через конкретный Python:

```
```powershell
```

```
C:\Python314\python.exe H:\XPE\tui\app.py
```

```
```
```

Но сначала выполните и пришлите результат двух команд:

```
```powershell
```

```
Test-Path C:\Python314\python.exe
```

```
Get-Command python -ErrorAction SilentlyContinue
```

```
```
```

С высокой вероятностью нужно будет использовать `C:\Python314\python.exe`, а не `py`.

-----

User prompt 25 of 25 - 9/8/2026, 11:03:02 PM:

PowerShell 7.6.0

A new PowerShell stable release is available: v7.6.5

Upgrade now, or check out the release page at:

<https://aka.ms/PowerShell-Release?tag=v7.6.5>

```
PS C:\Users\RJ> cd H:\XPE\Models\qwen-uncensored
```

```
PS H:\XPE\Models\qwen-uncensored> cd ../..
```

```
PS H:\XPE> py -m pip install textual
```

```
Can't find a default Python.
```

```
PS H:\XPE> cd C:\Python314
```

```
PS C:\Python314> py -m pip install textual
```

Can't find a default Python.

PS C:\Python314> Test-Path C:\Python314\python.exe

True

PS C:\Python314> C:\Python314\python.exe -m pip install textual

Defaulting to user installation because normal site-packages is not writeable

Collecting textual

Downloading textual-8.2.8-py3-none-any.whl.metadata (9.1 kB)

Collecting markdown-it-py>=2.1.0 (from markdown-it-py[linkify]>=2.1.0->textual)

Using cached markdown\_it\_py-4.2.0-py3-none-any.whl.metadata (7.4 kB)

Collecting mdit-py-plugins (from textual)

Downloading mdit\_py\_plugins-0.6.1-py3-none-any.whl.metadata (2.9 kB)

Requirement already satisfied: platformdirs<5,>=3.6.0 in

C:\Users\RJ\AppData\Roaming\Python\Python314\site-packages (from textual) (4.10.0)

Collecting pygments<3.0.0,>=2.19.2 (from textual)

Using cached pygments-2.21.0-py3-none-any.whl.metadata (2.5 kB)

Collecting rich>=14.2.0 (from textual)

Downloading rich-15.0.0-py3-none-any.whl.metadata (18 kB)

Requirement already satisfied: typing-extensions<5.0.0,>=4.4.0 in

C:\Users\RJ\AppData\Roaming\Python\Python314\site-packages (from textual) (4.15.0)

Collecting mdurl~=0.1 (from markdown-it-py>=2.1.0->markdown-it-py[linkify]>=2.1.0->textual)

Using cached mdurl-0.1.2-py3-none-any.whl.metadata (1.6 kB)

Collecting linkify-it-py<3,>=1 (from markdown-it-py[linkify]>=2.1.0->textual)

Downloading linkify\_it\_py-2.2.0-py3-none-any.whl.metadata (8.6 kB)

Downloading textual-8.2.8-py3-none-any.whl (731 kB)

---

—— 731.4/731.4 kB 1.8 MB/s 0:00:00

Using cached pygments-2.21.0-py3-none-any.whl (1.3 MB)

Using cached markdown\_it\_py-4.2.0-py3-none-any.whl (91 kB)

Using cached mdurl-0.1.2-py3-none-any.whl (10.0 kB)

Downloading linkify\_it\_py-2.2.0-py3-none-any.whl (21 kB)

Downloading rich-15.0.0-py3-none-any.whl (310 kB)

Downloading mdit\_py\_plugins-0.6.1-py3-none-any.whl (66 kB)

Installing collected packages: pygments, mdurl, linkify-it-py, markdown-it-py, rich, mdit-py-plugins, textual

---

—— 0/7 [pygments] WARNING: The script pygmentize.exe is installed in 'C:\Users\RJ\AppData\Roaming\Python\Python314\Scripts' which is not on PATH.

Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.

---

—— 3/7 [markdown-it-py] WARNING: The script markdown-it.exe is installed in 'C:\Users\RJ\AppData\Roaming\Python\Python314\Scripts' which is not on PATH.

Consider adding this directory to PATH or, if you prefer to suppress this warning, use --no-warn-script-location.

Successfully installed linkify-it-py-2.2.0 markdown-it-py-4.2.0 mdit-py-plugins-0.6.1 mdurl-0.1.2 pygments-2.21.0 rich-15.0.0 textual-8.2.8

[notice] A new release of pip is available: 26.1.2 -> 26.2.1

[notice] To update, run: python.exe -m pip install --upgrade pip

PS C:\Python314>